

Hands on DAY-35 – Keerthivasan R

Problem Statement

Many businesses need a system to manage product information such as product name, price, category, and stock details. In traditional applications, data is often tightly coupled with the user interface, making it difficult to share data with other applications like mobile apps or third-party systems.

There is a need for a centralized backend system that can expose data securely and allow different applications to access and manage it through standard HTTP methods.

The goal of this project is to develop a **RESTful Web API** using **ASP.NET Core** that performs CRUD (Create, Read, Update, Delete) operations on product data.

Objectives

- Build a Web API using ASP.NET Core
- Implement RESTful endpoints
- Perform CRUD operations
- Connect the API to a SQL Server database using Entity Framework Core
- Test API endpoints using Postman or Swagger

Expected Outcome

The final system should:

- Allow clients to retrieve all products
- Add new products
- Update existing products
- Delete products
- Return data in JSON format

CODE:

Modles/ Product.cs

```
using System.ComponentModel.DataAnnotations.Schema;

namespace ProductManagementAPI.Models
{
    public class Product
    {
        public int Id { get; set; }
        public string Name { get; set; } = string.Empty;

        [Column(TypeName = "decimal(18,2)")]
        public decimal Price { get; set; }

        public string Category { get; set; } = string.Empty;
        public int StockAvailable { get; set; }
    }
}
```

Data / ApplicationDbContext.cs

```
using Microsoft.EntityFrameworkCore;
using ProductManagementAPI.Models;

namespace ProductManagementAPI.Data
{
    public class ApplicationDbContext : DbContext
    {
        public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
            : base(options)
        {
        }

        public DbSet<Product> Products { get; set; }

        // This method is where you configure the model details
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            base.OnModelCreating(modelBuilder);

            // Explicitly set the precision and scale for the Price property
            modelBuilder.Entity<Product>()
                .Property(p => p.Price)
                .HasPrecision(18, 2);
        }
    }
}
```

appsettings.json

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "ConnectionStrings": {
    "DefaultConnection":
"Server=(localdb)\\mssqllocaldb;Database=ProductDB;Trusted_Connection=True;MultipleActiveResultSets=true;TrustServerCertificate=True"
  },
  "AllowedHosts": "*"
}
```

Program.cs

```
using Microsoft.EntityFrameworkCore;
using ProductManagementAPI.Data;

var builder = WebApplication.CreateBuilder(args);
// Add services to the container.
// Register DbContext with SQL Server
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));
builder.Services.AddControllers();
// Learn more about configuring Swagger/OpenAPI at https://aka.ms/aspnetcore/swashbuckle
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();
var app = builder.Build();
// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}
app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();
app.Run();
```

```

using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using ProductManagementAPI.Data;
using ProductManagementAPI.Models;
namespace ProductManagementAPI.Controllers{
    [Route("api/[controller]")]
    [ApiController]
    public class ProductsController : ControllerBase    {
        private readonly ApplicationDbContext _context;

        public ProductsController(ApplicationDbContext context)        {
            _context = context;
        }
        // GET: api/Products
        [HttpGet]
        public async Task<ActionResult<IEnumerable<Product>>> GetProducts()        {
            return await _context.Products.ToListAsync();
        }
        // GET: api/Products/5
        [HttpGet("{id}")]
        public async Task<ActionResult<Product>> GetProduct(int id)        {
            var product = await _context.Products.FindAsync(id);

            if (product == null)        {
                return NotFound();
            }
            return product;
        }
        // PUT: api/Products/5
        [HttpPut("{id}")]
        public async Task<IActionResult> PutProduct(int id, Product product)        {
            if (id != product.Id)        {
                return BadRequest();
            }
            _context.Entry(product).State = EntityState.Modified;
            try        {
                await _context.SaveChangesAsync();
            }
            catch (DbUpdateConcurrencyException)        {
                if (!ProductExists(id))        {
                    return NotFound();
                }
                else        {
                    throw;
                }
            }
            return NoContent();
        }
        // POST: api/Products
        [HttpPost]
        public async Task<ActionResult<Product>> PostProduct(Product product)        {
            _context.Products.Add(product);
            await _context.SaveChangesAsync();

            return CreatedAtAction("GetProduct", new { id = product.Id }, product);
        }
        // DELETE: api/Products/5
        [HttpDelete("{id}")]
        public async Task<IActionResult> DeleteProduct(int id)        {
            var product = await _context.Products.FindAsync(id);
            if (product == null)        {
                return NotFound();
            }
            _context.Products.Remove(product);
            await _context.SaveChangesAsync();
            return NoContent();
        }
        private bool ProductExists(int id)        {
            return _context.Products.Any(e => e.Id == id);
        }
    }
}

```

OUTPUT :

GET

/api/Products

Parameters

No parameters

Cancel

Execute

Clear

Responses

curl

```
curl -X 'GET' \
  'https://localhost:7246/api/Products' \
  -H 'accept: application/json'
```

Request URL

https://localhost:7246/api/Products

Server response

Code	Details
200	Response body <pre>{ "id": 1, "name": "Samsung Laptop", "price": 1299.99, "category": "Electronics", "stockAvailable": 20 }</pre> Response headers <pre>content-type: application/json; charset=utf-8 date: Tue, 02 Apr 2024 14:15:55 GMT server: Kestrel</pre>

Responses

Code	Description	Links
200	OK	No links

Media type

application/json

Content Accept header

Example Value | Schema

```
{
  "id": 0,
  "name": "string",
  "price": 0,
  "category": "string",
  "stockAvailable": 0
}
```

GET

/api/Products/{id}

Parameters

Cancel

Name	Description
id * required	
integer(int32)	1
(path)	

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'https://localhost:7346/api/Products/1' \
  -H 'accept: text/plain'
```

Request URL

https://localhost:7346/api/Products/1

Server response

Code	Details
200	Response body <pre>{ "id": 1, "name": "Laptop", "price": 999.99, "category": "Electronics", "stockavailable": 20 }</pre> Download Response headers <pre>content-type: application/json; charset=utf-8 date: Tue, 07 Apr 2025 16:34:59 GMT server: Restrel</pre>

Responses

Code	Description	Links
200	OK	No links

Media type

text/plain

Controls Accept header

Example Value | Schema

```
{
  "id": 0,
  "name": "string",
  "price": 0,
  "category": "string",
  "stockavailable": 0
}
```

DELETE

/api/Products/{id}

^

Parameters

Cancel

Name	Description
id * required	
integer(\$int32)	1
(path)	

Execute

Clear

Responses

Curl

curl -X 'DELETE' \n'https://localhost:7246/api/Products/1' \n-H 'accept: */*'

Request URL

https://localhost:7246/api/Products/1

Server response

Code	Details
204	Response headers
Undocumented	date: Tue, 07 Apr 2025 16:36:07 GMT\nserver: Restrel

Responses

Code	Description	Links
200	OK	No links

-H 'Content-Type: application/json' \n-d '{\n "id": 1,\n "name": "Gaming Laptop",\n "price": 1299.99,\n "category": "Electronics",\n "stockAvailable": 25\n}'

Request URL

https://localhost:7246/api/Products/1

Server response

Code	Details
204	Response headers
Undocumented	date: Tue, 07 Apr 2025 16:35:29 GMT\nserver: Restrel

Responses

Code	Description	Links
200	OK	No links

POST

/api/Products

Parameters

No parameters

Request body

application/json

{
 "name": "Laptop",
 "price": 999.99,
 "category": "Electronics",
 "stockAvailable": 25
}

Execute

Clear

Responses

Curl

curl -X 'POST' \
'https://localhost:7246/api/Products' \
-H 'accept: text/plain' \
-H 'Content-Type: application/json' \
-d '{
 "name": "Laptop",
 "price": 999.99,
 "category": "Electronics",
 "stockAvailable": 25
}'

Request URL

https://localhost:7246/api/Products

Server response

Code	Details
201	Response body { "id": 1, "name": "Laptop", "price": 999.99, "category": "Electronics", "stockAvailable": 25 } Download Response headers content-type: application/json; charset=utf-8 date: Tue, 07 Apr 2025 16:34:10 GMT location: https://localhost:7246/api/Products/1 server: Kestrel

Responses

Code	Description	Links
200	OK Media type text/plain Controls Accept header. Example Value Schema { "id": 0, "name": "string", "price": 0, "category": "string", "stockAvailable": 0 }	No links